

Reproducibility

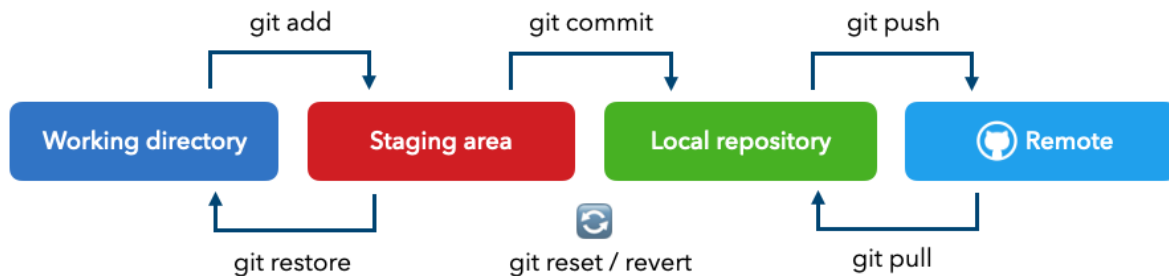
Versioning Code, Data and Models: Versioning code, data and models to ensure reproducible behavior in ML systems.

Intuition: how to version our code, data and models to ensure reproducible behavior in our ML systems. It's imperative that we can reproduce our results and track changes to our system so we can debug and improve our application. Without it, it would be difficult to share our work, recreate our models in the event of system failures and fall back to previous versions in the event of regressions.

Code

To version our code, we'll be using [git](#), which is a widely adopted version control system. In fact, when we clone any repository, we pull code from a git repository.

We can then make changes to the code and Git, which is running locally on our computer, will keep track of our files and their versions as we `add` and `commit` our changes. But it's not enough to just version our code locally, we need to `push` our work to a central location that can be `pulled` by us and others we want to grant access to. This is where remote repositories like [GitHub](#), [GitLab](#), [BitBucket](#), etc. provide a remote location to hold our versioned code in.

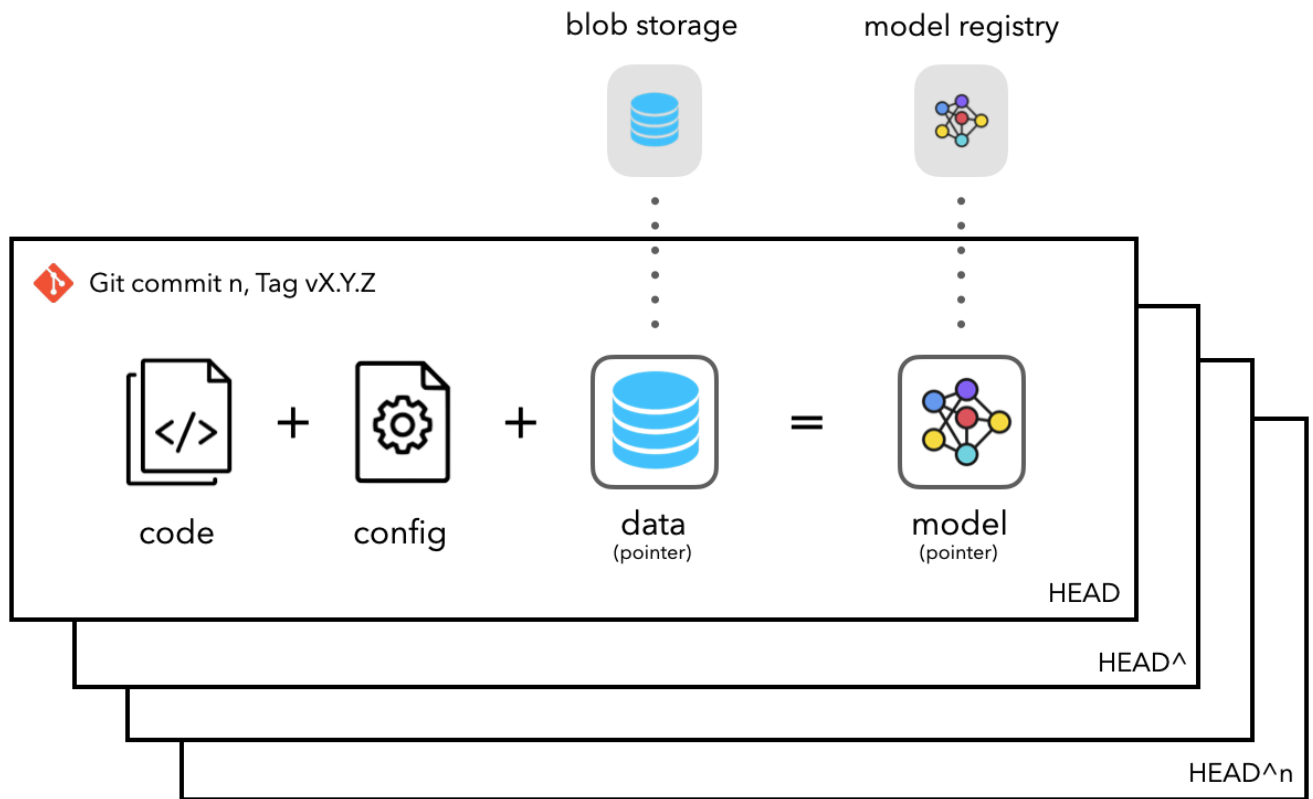


Here's a simplified workflow for how we version our code using GitHub:

```
[make changes to code]
git add .
git commit -m "message"
git push origin <branch-name>
```

Artifacts

While Git is ideal for saving our code, it's not ideal for saving artifacts like our datasets (especially unstructured data like text or images) and models. Also, recall that Git stores every version of our files and so large files that change frequently can very quickly take up space. So instead, it would be ideal if we could save locations (pointers) to these large artifacts in our code as opposed to the artifacts themselves. This way, we can version the locations of our artifacts and pull them as they're needed.



Data

While we're saving our dataset on GitHub for easy course access (and because our dataset is small), in a production setting, we would use a remote blob storage like S3 or a data warehouse like Snowflake. There are also many tools available for versioning our data, such as [GitLFS](#), [Dolt](#), [Pachyderm](#), [DVC](#), etc. With any of these solutions, we would be pointing to our remote storage location and versioning the pointer locations (ex. S3 bucket path) to our data instead of the data itself.

Models

And similarly, we currently store our models locally where the *MLflow* artifact and backend store are local directories.

```
# Config MLflow
MODEL_REGISTRY = Path("/tmp/mlflow")
Path(MODEL_REGISTRY).mkdir(parents=True, exist_ok=True)
MLFLOW_TRACKING_URI = "file://" + str(MODEL_REGISTRY.absolute())
mlflow.set_tracking_uri(MLFLOW_TRACKING_URI)
print (mlflow.get_tracking_uri())
```

In a production setting, these would be remote such as S3 for the artifact store and a database service (ex. [PostgreSQL RDS](#)) as our backend store. This way, our models can be versioned and others, with the appropriate access credentials, can pull the model artifacts and deploy them.